

Ejercicios FreeRTOS

1. Generar tareas.

Dado el programa:

https://drive.google.com/file/d/1qvaH1Mi5HkiDBo5ucjlxN5RwwL_K0anea/view?usp=sharing

que hace parpadear dos leds controlados por dos pulsadores.

Se pide:

- Haga una copia de este programa en un workspace vacío.
- Implemente la misma funcionalidad del programa, pero utilizando FreeRTOS. Es decir, el led de la placa y los dos leds adicionales (en C2 y C3 en el programa compartido, pero puede conectarlos de la manera que considere más conveniente) que parpaddeen con un período de 0.25s.
- Utilizar los pulsadores en B1 (en la placa de desarrollo) y B2 (en C0, pero puede conectarlo de la manera que considere más conveniente) para controlar los leds adicionales. Si los pulsadores están abiertos los leds parpaddeen, si están presionados, no.
- El LED de la placa debe parpadear continuamente con un período de 250 ms. Los otros dos LEDs también deben parpadear con un período de 250 ms mientras sus pulsadores asociados no estén presionados.
- Se sugiere utilizar tres tareas (una para el led de la placa que parpadea libremente y dos más para los leds adicionales).
- Se sugiere activar la resistencia de pull-up interna para el pulsador adicional o conectar una resistencia de pull-up que genere una corriente menor a 1 mA con el pulsador presionado.
- Se sugiere utilizar las funciones provistas en los archivos “interfaz_tp.h” e “interfaz_tp.c” para usar los leds y pulsadores adicionales:
 - void setear_led(int n_led, int estado). Enciende el led n_led si estado es verdadero sino lo apaga.
 - void invertir_led(int n_led). Invierte el estado del led n_led.
 - int pulsador_activo(int n_pulsador). Devuelve un valor distinto de cero si el pulsador n_pulsador está presionado (sin antirrebote).

2. Generar tareas (2).

Implementar el programa del punto anterior, pero utilizando código único para las tareas de los leds adicionales. Utilice una única función de tarea para ambos LEDs y pase mediante el parámetro pvParameters la información necesaria para distinguir cada instancia.

3. Prioridades.

Implementar una aplicación que genere tres tareas de FreeRTOS. **tarea_pulsadores**, **tarea_led1**, **tarea_led2**. Que se comporten de la siguiente manera:

- Cuando el sistema inicia, **tarea_led1** y **tarea_led2** tienen prioridad `tskIDLE_PRIORITY+1` ambas y **tarea_pulsadores** tiene prioridad `tskIDLE_PRIORITY+3`.
- La tarea **tarea_pulsadores** funciona de la siguiente manera. Cuando se presiona el pulsador B1 pone a **tarea_led1** con prioridad `tskIDLE_PRIORITY+2` y deja a **tarea_led2** con prioridad `tskIDLE_PRIORITY+1`, si se vuelve a presionar B1 **tarea_led1** pasa a tener prioridad `tskIDLE_PRIORITY+1` y **tarea_led2** pasa a tener prioridad `tskIDLE_PRIORITY+2`, si se presiona otra vez, vuelve a la primera condición y así indefinidamente.
- **tarea_led1** debe encender el led 0 y apagar el led 1 sin ejecutar funciones bloqueantes (`vTaskDelay()`, `xSemaphoreTake()` con timeout distinto de cero, etc.) .
- **tarea_led2** debe encender el led 1 y apagar el led 0 sin ejecutar funciones bloqueantes (`vTaskDelay()`, `xSemaphoreTake()` con timeout distinto de cero, etc.).
- Implemente la solución.
- Justifique los patrones de leds según presiona B1.

4. Semáforos (mutua exclusión).

Implementar una aplicación que genere dos tareas de FreeRTOS. **tarea_rojo**, **tarea_verde**, Que se comporten de la siguiente manera:

- Cuando el sistema inicia, **tarea_rojo** y **tarea_verde** tienen prioridad `tskIDLE_PRIORITY+1`.
- **tarea_rojo**, utiliza el led 0 y el pulsador de B1, mientras que **tarea_verde** el led 1 y el pulsador de B2.
- Ambas tareas deben comportarse de la misma manera. Las tareas intentan tomar el mutex **sem_mutex** y la tarea que lo toma enciende el led y lo mantiene encendido hasta que se presiona el pulsador asociado a la misma. Después, apaga el led, libera el mutex, ejecuta `vTaskDelay(1 ms)` y vuelve a intentar tomarlo.
- Implemente la solución.
- Describa el funcionamiento del programa, ¿Es esperable? ¿Cuántos leds pueden estar encendidos a la vez?

5. Semáforos (binario).

Implementar una aplicación que genere dos tareas de FreeRTOS. **tarea_pulsador** y **tarea_led**, Que se comporten de la siguiente manera:

- Cuando el sistema inicia, **tarea_pulsador** y **tarea_led** tienen la misma prioridad. También crea el semáforo binario **sem_led**.
- **tarea_led**, utiliza el led 0 y espera que por **sem_led** infinitamente. Cuando lo consigue invierte el led 0. **No libera el semáforo sem_led**.

- **tarea_pulsador.** Se encarga de realizar el antirrebote del pulsador B1 y medir el tiempo que permanece presionado (en ms). Luego de liberar el pulsador, la tarea deberá generar periódicamente eventos mediante `xSemaphoreGive()` sobre `sem_led`. El período utilizado será igual al tiempo que estuvo presionado el pulsador, limitado a un mínimo de 50 ms y un máximo de 1 s. Es decir, si el tiempo medido es menor a 50 ms, el período será de 50 ms; si es mayor a 1 s, el período será de 1 s; y para valores intermedios, el período será igual al tiempo medido. La generación periódica continúa indefinidamente hasta que se produzca una nueva pulsación, momento en el cual deberá medirse nuevamente el tiempo presionado y actualizar el período correspondiente.
- Se recomienda implementar esta tarea mediante una máquina de estados y bloquearla periódicamente utilizando `vTaskDelay(pdMS_TO_TICKS(1))`. Puede resultar útil la función `xTaskGetTickCount()` para resolver esta tarea: <https://www.freertos.org/a00021.html#xTaskGetTickCount>.

6. Colas de mensajes.

Implementar la aplicación del punto anterior utilizando una cola de mensajes en lugar del semáforo binario. Enviar mediante la cola un mensaje cada vez que deba invertirse el LED.

7. Notificaciones de tarea (Task Notifications).

Implementar una aplicación en FreeRTOS que genere una tarea que lea los pulsadores B1 y B2 y que por medio de una notificación de tarea envíe un mensaje a dos tareas distintas, una asociada a cada pulsador. La tarea asociada al pulsador B1 envía por línea serie el mensaje "B1\r\n" y la tarea asociada al pulsador B2 envía el mensaje "B2\r\n".

8. Notificaciones de tarea (Task Notifications).

Implementar una solución para el problema anterior, pero utilizando solamente dos tareas y notificaciones de tareas con pasaje de valores para resolver el mensaje. Utilice el valor de la notificación para distinguir si el evento fue generado por B1 o B2.

9. Semáforos binarios en interrupción.

Resuelva el problema de semáforos binarios (Problema 5) pero utilizando el pulsador activado por la interrupción de pines. Cambie el pulsador al que le sea más conveniente y también puede cambiar las tareas a su conveniencia.

10. Interrupciones de puerto serie.

Configure la UART que le sea conveniente en formato 9600bps-8-N-1 y utilícela por interrupciones tanto para transmitir como para recibir datos. El comportamiento debe ser el siguiente:

- Recibir por línea serie una cadena de caracteres terminada en \n que tiene el siguiente formato (los espacios indicados son obligatorios):
 - o LED 0 0\n para apagar el led 0
 - o LED 0 1\n para encender el led 0
 - o LED 1 0\n para apagar el led 1
 - o LED 1 1\n para encender el led 1
 - o A todos estos mensajes el equipo contesta OK\n mientras que a cualquier otro mensaje contesta ERROR.
- Al presionar el pulsador B1 enviar por línea serie B1\n.
- Al presionar el pulsador B2 enviar por línea serie B2\n.
- Se recomienda utilizar una tarea para recibir los caracteres serie y procesar la detección de líneas, así como otra tarea para transmitir los caracteres serie. El resto de las tareas necesarias queda a su conveniencia.